

Các thuật toán tìm kiếm siêu tham số trong hệ thống HAutoML

Nhóm nghiên cứu AutoML — OptiVisionLab

Ngày 27 tháng 5 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày chi tiết cơ sở toán học, giả mã, phân tích độ phức tạp và chiến lược triển khai của các thuật toán tìm kiếm siêu tham số được sử dụng trong hệ thống HAutoML: **Grid Search** (tìm kiếm vét cạn), **Bayesian Optimization** (tối ưu hóa Bayesian sử dụng Gaussian Process), và **Genetic Algorithm** (thuật toán di truyền). Mỗi thuật toán được phân tích từ lý thuyết đến triển khai thực tế, kèm theo so sánh định lượng về hiệu suất và phạm vi ứng dụng phù hợp.

Mục lục

1	Giới thiệu	3
1.1	Bài toán tối ưu hóa siêu tham số	3
1.2	Các chỉ số đánh giá	3
1.2.1	Bài toán phân loại (Classification)	3
1.2.2	Bài toán hồi quy (Regression)	4
2	Grid Search — Tìm kiếm vét cạn	5
2.1	Định nghĩa	5
2.2	Giả mã	5
2.2.1	Các thủ tục phụ trợ	5
2.2.2	Thuật toán chính	10
2.3	Phân tích độ phức tạp	10
2.4	Tối ưu hóa triển khai trong HAutoML	11
3	Bayesian Optimization — Tối ưu hóa Bayesian	11
3.1	Nguyên lý hoạt động	11
3.2	Gaussian Process	11
3.3	Hàm thu nhận (Acquisition Functions)	12
3.3.1	Expected Improvement (EI)	12
3.3.2	Probability of Improvement (PI)	12
3.3.3	Lower Confidence Bound (LCB)	12
3.4	Xử lý mất cân bằng lớp	12
3.5	Phép nghịch đảo dấu trong hàm mục tiêu	13
3.6	Giả mã	13
3.6.1	Các thủ tục phụ trợ	13

3.6.2	Thuật toán chính	16
3.7	Phân tích độ phức tạp	16
4	Genetic Algorithm — Thuật toán di truyền	17
4.1	Tổng quan	17
4.2	Mã hóa cá thể	17
4.3	Tournament Selection — Chọn lọc đấu trường	17
4.4	BLX- α Crossover — Lai ghép	17
4.5	Adaptive Gaussian Mutation — Đột biến thích nghi	18
4.6	Elitism	18
4.7	Độ đa dạng quần thể	18
4.7.1	Phương pháp $O(n)$ — Variance-based (quần thể lớn, $P \geq 20$) . . .	18
4.7.2	Phương pháp $O(n^2)$ — Pairwise (quần thể nhỏ, $P < 20$)	18
4.8	Diversity Injection — Tiêm đa dạng	19
4.9	Giả mã	19
4.9.1	Các thủ tục phụ trợ	19
4.9.2	Thuật toán chính	26
4.10	Phân tích độ phức tạp	26
5	So sánh các thuật toán	27
5.1	Bảng so sánh chi tiết	27
5.1.1	Phân tích chi tiết các tiêu chí	27
5.2	Hướng dẫn chọn thuật toán	28
6	Cơ chế quản lý thời gian chung	28
6.1	Proactive Time Limit	28
6.2	Early Stopping vs. Time Limit	28
A	Bảng giải nghĩa thuật ngữ và siêu tham số	30
A.1	Bảng giải nghĩa các thuật ngữ chung	30
A.2	Bảng giải nghĩa các tham số cấu hình hệ thống và mô hình	33

1 Giới thiệu

1.1 Bài toán tối ưu hóa siêu tham số

Cho một mô hình học máy $\mathcal{M}$ với tập siêu tham số $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_m)$ thuộc không gian tìm kiếm $\Theta = \Theta_1 \times \Theta_2 \times \dots \times \Theta_m$, bài toán tối ưu hóa siêu tham số được phát biểu như sau:

$$\boldsymbol{\theta}^* = \operatorname{argmax}_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\mathcal{M}_{\boldsymbol{\theta}}, \mathcal{D}) \quad (1)$$

trong đó $\mathcal{L}$ là hàm mục tiêu (ví dụ: accuracy, F1-score) được ước lượng trên tập dữ liệu $\mathcal{D}$ thông qua kiểm định chéo k -fold (cross-validation):

$$\mathcal{L}(\mathcal{M}_{\boldsymbol{\theta}}, \mathcal{D}) = \frac{1}{k} \sum_{i=1}^k \operatorname{Score}(\mathcal{M}_{\boldsymbol{\theta}}^{(\mathcal{D} \setminus F_i)}, F_i) \quad (2)$$

với $F_1, F_2, \dots, F_k$ là k fold rời rạc của $\mathcal{D}$, và $\mathcal{M}_{\boldsymbol{\theta}}^{(\mathcal{D} \setminus F_i)}$ là mô hình được huấn luyện trên $\mathcal{D} \setminus F_i$ với siêu tham số $\boldsymbol{\theta}$.

1.2 Các chỉ số đánh giá

1.2.1 Bài toán phân loại (Classification)

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (3)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (5)$$

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{i=1}^K \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i} \quad (7)$$

với K là số lớp (classes).

1.2.2 Bài toán hồi quy (Regression)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (8)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (9)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (10)$$

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (11)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (12)$$

2 Grid Search — Tìm kiếm vét cạn

2.1 Định nghĩa

Định nghĩa 2.1 (Grid Search). Cho không gian siêu tham số rời rạc $\Theta = V_1 \times V_2 \times \dots \times V_m$, trong đó mỗi $V_j = \{v_j^{(1)}, v_j^{(2)}, \dots, v_j^{(n_j)}\}$ là tập hữu hạn các giá trị thử nghiệm cho siêu tham số θ_j . Grid Search duyệt qua **tất cả** các tổ hợp tham số trong Θ và chọn tổ hợp tối ưu:

$$\theta^* = \operatorname{argmax}_{\theta \in \Theta} \mathcal{L}(\mathcal{M}_\theta, \mathcal{D}) \quad (13)$$

Tổng số tổ hợp cần đánh giá:

$$N = \prod_{j=1}^m |V_j| = \prod_{j=1}^m n_j \quad (14)$$

2.2 Giả mã

Thuật toán Grid Search trong HAutoML sử dụng nhiều thủ tục phụ trợ bao gồm: lựa chọn backend thực thi (Thuật toán 1), tạo khóa hash cho bộ nhớ đệm (Thuật toán 2), đánh giá tổ hợp tham số đơn lẻ (Thuật toán 3), đánh giá theo batch (Thuật toán 4), kiểm tra dừng sớm (Thuật toán 5), và kiểm tra thời gian proactive (Thuật toán 6).

2.2.1 Các thủ tục phụ trợ

Algorithm 1 Lựa chọn backend thực thi tối ưu (SELECTOPTIMALBACKEND)

Require: N (số tổ hợp tham số), $|\mathcal{D}|_{\text{size}}$ (kích thước dữ liệu = số mẫu $\times$ số đặc trưng),
auto_select_backend

Ensure: backend $\in \{\text{threading}, \text{loky}\}$

```

1: if auto_select_backend = false then
2:   return loky                                ▷ Mặc định khi không auto select
3: end if
4: if  $N \leq 4$  then
5:   return threading                            ▷ Overhead thấp cho ít tổ hợp
6: end if
7:  $W \leftarrow |\mathcal{D}|_{\text{size}} \times N$                 ▷ Tổng workload
8: if  $W > 10^6$  then
9:   return loky                                ▷ Multiprocessing cho workload lớn
10: end if
11: return threading                             ▷ Threading cho trường hợp trung bình

```

Algorithm 2 Tạo khóa hash cho bộ nhớ đệm (GETPARAMSHASH)

Require: Mô hình $\mathcal{M}$, tổ hợp tham số θ

Ensure: h (chuỗi hash MD5 duy nhất)

- 1: $\text{class_info} \leftarrow \mathcal{M}.\text{module} \parallel \mathcal{M}.\text{class_name}$
 - 2: $\text{base_params} \leftarrow \text{Sorted}(\mathcal{M}.\text{get_params}())$
 - 3: $\text{input} \leftarrow \text{class_info} \parallel \text{base_params} \parallel \text{Sorted}(\theta)$
 - 4: **return** MD5(input)
-

Algorithm 3 Đánh giá tổ hợp tham số đơn lẻ có bộ nhớ đệm (EVALUATESINGLE)

Require: θ , mô hình $\mathcal{M}$, dữ liệu $(\mathbf{X}, \mathbf{y})$, k -fold CV, scoring (dict các metric), cache, cache_enabled

Ensure: $\text{result} = \{\text{test_scores}, \text{params}, \text{fit_time}, \text{score_time}\}$

- 1: $h \leftarrow \text{GETPARAMSHASH}(\mathcal{M}, \theta)$
- 2: **if** cache_enabled **và** $h \in \text{cache}$ **then**
- 3: **return** $\text{cache}[h]$ ▷ Cache hit — bỏ qua đánh giá
- 4: **end if**
- 5: $\mathcal{M}.\text{set_params}(\theta)$
- 6: $\text{cv_scores} \leftarrow \text{CROSSVALIDATE}(\mathcal{M}, \mathbf{X}, \mathbf{y}, \text{cv} = k, \text{scoring}, \text{n_jobs} = 1, \text{error_score} = \text{raise})$
▷ $\text{n_jobs} = 1$ vì song song hóa ở mức batch (Thuật toán 4)
- 7: $\text{result} \leftarrow \{\text{test_scores} : \text{cv_scores}, \text{params} : \theta, \text{fit_time} : \overline{t_{\text{fit}}}, \text{score_time} : \overline{t_{\text{score}}}\}$
- 8: **if** cache_enabled **then**
- 9: $\text{cache}[h] \leftarrow \text{result}$
- 10: **end if**
- 11: **return** result

// Nếu xảy ra lỗi: trả về result với test_scores = None, error = mô tả lỗi

Algorithm 4 Đánh giá batch tổ hợp tham số — song song hoặc tuần tự (EVALUATEPARAMSBATCH)

Require: Danh sách batch = $[\theta_1, \dots, \theta_b]$, mô hình $\mathcal{M}$, dữ liệu $(\mathbf{X}, \mathbf{y})$, k -fold, scoring

Require: parallel_enabled, n_jobs, model_copies

Ensure: Danh sách results

```

1: if parallel_enabled và |batch| > 1 then
2:   if n_jobs = -1 then
3:     n_jobs ← CPU_COUNT()
4:   end if
5:   if n_jobs > 1 then
6:     // Tạo trước bản sao mô hình để quản lý bộ nhớ
7:     n_actual ← min(n_jobs, |batch|)
8:     if |model_copies| < n_actual then
9:       model_copies ← [DEEPCOPY( $\mathcal{M}$ )]i=1n_actual
10:    end if
11:    backend ← SELECTOPTIMALBACKEND(|batch|, | $\mathbf{X}$ |rows × | $\mathbf{X}$ |cols)
12:    return PARALLEL(n_jobs, backend) [EVALUATESINGLE( $\theta_i$ , model_copies[i mod
    n_actual], ...)]
13:  end if
14: end if
// Fallback: đánh giá tuần tự
14: return [EVALUATESINGLE( $\theta_i$ ,  $\mathcal{M}$ , ...) cho mỗi  $\theta_i \in$  batch]

```

Algorithm 5 Kiểm tra điều kiện dừng sớm (CHECKEARLYSTOPPING)

Require: s^* (điểm tốt nhất), $\mathcal{H}$ (lịch sử best scores), $T_{\max}$

Require: es_enabled , s_{target} (ngưỡng score), n_{best} , $n_{\text{no_improve}}$

Ensure: $\text{should_stop} \in \{\text{true}, \text{false}\}$

// Early stopping bị vô hiệu hóa khi có time limit

1: **if** $T_{\max} \neq \text{None}$ **then**

2: **return** false

▷ Ưu tiên time limit

3: **end if**

4: **if** $\text{es_enabled} = \text{false}$ **then**

5: **return** false

6: **end if**

// Kiểm tra ngưỡng score mục tiêu

7: **if** $s_{\text{target}} \neq \text{None}$ và $s^* \geq s_{\text{target}}$ **then**

8: $n_{\text{good}} \leftarrow |\{s \in \mathcal{H} : s \geq s_{\text{target}}\}|$

9: **if** $n_{\text{good}} \geq \max(1, n_{\text{best}})$ **then**

10: **return** true

▷ Đạt đủ số kết quả tốt

11: **end if**

12: **end if**

// Kiểm tra không cải thiện liên tiếp

13: **if** $n_{\text{no_improve}} \neq \text{None}$ và $|\mathcal{H}| \geq n_{\text{no_improve}}$ **then**

14: $\text{recent} \leftarrow \mathcal{H}[-n_{\text{no_improve}} :]$

15: **if** $|\text{Set}(\text{recent})| = 1$ **then**

16: **return** true

▷ Không cải thiện

17: **end if**

18: **end if**

19: **return** false

Algorithm 6 Kiểm tra thời gian proactive — EMA (SHOULDSTARTNEXTITERATION)

Require: t_{iter} (thời gian iteration vừa hoàn thành, có thể None), T_{max} , t_{start} , $\hat{t}_{\text{ema}}$

Ensure: $\text{should_continue} \in \{\text{true}, \text{false}\}$

```
1: if  $T_{\text{max}} = \text{None}$  then
2:   return true                                     ▷ Không giới hạn thời gian
3: end if
4:  $t_{\text{elapsed}} \leftarrow \text{TIME}() - t_{\text{start}}$ 
5:  $t_{\text{remaining}} \leftarrow \max(0, T_{\text{max}} - t_{\text{elapsed}})$ 
6: if  $t_{\text{elapsed}} \geq T_{\text{max}}$  then
7:   return false                                   ▷ Đã vượt quá time limit
8: end if
   // Cập nhật EMA nếu có iteration_duration
9: if  $t_{\text{iter}} \neq \text{None}$  then
10:  if  $\hat{t}_{\text{ema}} = \text{None}$  then
11:     $\hat{t}_{\text{ema}} \leftarrow t_{\text{iter}}$ 
12:  else
13:     $\hat{t}_{\text{ema}} \leftarrow 0.7 \cdot t_{\text{iter}} + 0.3 \cdot \hat{t}_{\text{ema}}$ 
14:  end if
15: end if
   // Kiểm tra proactive: ước tính iteration tiếp theo
16: if  $\hat{t}_{\text{ema}} \neq \text{None}$  và  $1.2 \cdot \hat{t}_{\text{ema}} > t_{\text{remaining}}$  then
17:   return false                                   ▷ Dừng proactive — safety factor  $1.2\times$ 
18: end if
19: return true
```

2.2.2 Thuật toán chính

Algorithm 7 Grid Search với tối ưu hóa tài nguyên trong HAutoML

Require: Mô hình $\mathcal{M}$, lưới tham số Θ (dict hoặc list-of-dicts), dữ liệu $\mathcal{D} = (\mathbf{X}, \mathbf{y})$, số fold k
Require: Cấu hình: `parallel_evaluation`, `batch_size`, $T_{\max}$ (tùy chọn), `scoring` (dict các metric), `metric_sort`
Require: Early stopping: `es_enabled`, s_{target} , n_{best} , $n_{\text{no_improve}}$
Ensure: θ^* , s^* , `all_scores*`, `cv_results`, `time_limit_reached`

```

// Khởi tạo
1: STARTTIMER();  $\hat{t}_{\text{ema}} \leftarrow \text{None}$ ;  $s^* \leftarrow -\infty$ ;  $\theta^* \leftarrow \emptyset$ 
2: cache  $\leftarrow \emptyset$ ; model_copies  $\leftarrow []$ ;  $\mathcal{H} \leftarrow []$ 
   // Chuẩn hóa param_grid và sinh tất cả tổ hợp
3:  $\Theta_{\text{list}} \leftarrow \text{NORMALIZEPARAMGRID}(\Theta)$  ▷ Chuyển về list-of-dicts
4:  $\mathcal{S} \leftarrow []$ 
5: for mỗi grid  $\in \Theta_{\text{list}}$  do
6:    $\mathcal{S} \leftarrow \mathcal{S} \cup \text{CARTESIANPRODUCT}(\text{grid})$  ▷ Sinh tổ hợp từ mỗi grid
7: end for
8:  $N \leftarrow |\mathcal{S}|$ 
9: if  $N = 0$  then
10:   return  $\emptyset, -\infty, \emptyset, \emptyset, \text{false}$ 
11: end if
   // Điều tiết kích thước batch
12: if  $T_{\max} \neq \text{None}$  then
13:    $b \leftarrow 1$  ▷ batch = 1 khi có giới hạn thời gian
14: else
15:    $b \leftarrow \text{batch\_size}$  ▷ Lấy từ tệp cấu hình hệ thống
16: end if
   // Vòng lặp chính theo batch
17:  $i \leftarrow 0$ ; early_stopped  $\leftarrow \text{false}$ ; time_stopped  $\leftarrow \text{false}$ 
18: while  $i < N$  do
19:   if SHOULDSTARTNEXTITERATION(None,  $T_{\max}$ , ...) = false then
20:     time_stopped  $\leftarrow \text{true}$ ; break
21:   end if
22:   batch  $\leftarrow \mathcal{S}[i : i + b]$ ;  $t_0 \leftarrow \text{TIME}()$ 
23:   results  $\leftarrow \text{EVALUATEPARAMSBATCH}(\text{batch}, \mathcal{M}, \mathbf{X}, \mathbf{y}, k, \text{scoring}, \dots)$ 
24:   SHOULDSTARTNEXTITERATION( $\text{TIME}() - t_0$ , ...) ▷ Cập nhật EMA
25:   for mỗi result  $\in \text{results}$  do
26:     if result.test_scores  $\neq \text{None}$  then
27:        $s \leftarrow \text{result.test\_scores}[\text{metric\_sort}]$ 
28:       if  $s \geq s^*$  then
29:          $s^* \leftarrow s$ ;  $\theta^* \leftarrow \text{result.params}$ ; all_scores*  $\leftarrow \text{result.means}$ 
30:       end if
31:     end if
32:     LOGEVALUATION( $\mathcal{M}$ , result.params, scores,  $i$ ,  $N$ )
33:   end for
34:    $\mathcal{H}.\text{append}(s^*)$  ▷ Cập nhật lịch sử best score
35:   if CHECKEARLYSTOPPING( $s^*$ ,  $\mathcal{H}$ ,  $T_{\max}$ , es_enabled,  $s_{\text{target}}$ ,  $n_{\text{best}}$ ,  $n_{\text{no\_improve}}$ ) then
36:     early_stopped  $\leftarrow \text{true}$ ; break
37:   end if
38:    $i \leftarrow i + b$ 
39: end while
   // Xây dựng cv_results_ và ranking
40: cv_results  $\leftarrow \text{BUILDCVRESULTS}(\text{all\_results}, \text{scoring}, \text{metric\_sort})$ 
41: for mỗi metric  $\in \text{Keys}(\text{scoring})$  do
42:   cv_results[rank_test_ || metric]  $\leftarrow \text{ARGSORT}(-\text{scores}) + 1$ 
43: end for
   // Xóa cache và chuyển đổi kiểu numpy  $\rightarrow$  Python gốc
44: Xóa cache, model_copies; Chuyển đổi numpy types
45: return  $\theta^*$ ,  $s^*$ , all_scores*, cv_results, time_stopped

```

2.3 Phân tích độ phức tạp

- **Thời gian:** $\mathcal{O}(k \cdot N \cdot T(\mathcal{D}, \theta))$, trong đó $N = \prod_{j=1}^m n_j$, k là số fold, $T(\mathcal{D}, \theta)$ là thời gian huấn luyện/đánh giá. Sự bùng nổ theo hàm mũ (N tăng theo $\prod n_j$) là rào cản kỹ thuật chính khi quy mô tham số gia tăng.
- **Không gian:** $\mathcal{O}(1)$ (ngoài cache tùy chọn).

2.4 Tối ưu hóa triển khai trong HAutoML

Hệ thống HAutoML nâng cấp Grid Search thông qua các giải pháp tối ưu hóa tài nguyên vật lý:

1. **Lưu trữ đệm (Caching)**: Cache kết quả theo $\text{Hash}(\mathcal{M}.\text{class}, \boldsymbol{\theta})$ để tránh đánh giá trùng lặp.
2. **Xử lý theo lô (Batch Processing)**: Khi $T_{\max}$ được xác lập, `batch_size` tự động chuyển về 1 để kiểm soát thời gian chính xác; ngược lại sử dụng giá trị từ tệp cấu hình.
3. **Đánh giá song song**: Kích hoạt khi `parallel_evaluation = true`. Nếu `false`, quy trình thực thi tuần tự ngay cả khi `n_jobs > 1`.
4. **Auto Backend Selection**: Tự động chọn `threading` (overhead thấp) hoặc `loky` (multiprocessing) dựa trên tổng workload $W = |\mathcal{D}| \times N$.
5. **Proactive Time Limit**: Sử dụng EMA (Exponential Moving Average) để ước tính thời gian batch tiếp theo và dừng sớm nếu vượt quá giới hạn:

$$\hat{t}_{\text{next}} = 0.7 \cdot t_{\text{current}} + 0.3 \cdot \hat{t}_{\text{prev}} \quad (15)$$

3 Bayesian Optimization — Tối ưu hóa Bayesian

3.1 Nguyên lý hoạt động

Chiến lược Bayesian Optimization [1] được định vị là giải pháp tối ưu cho các kịch bản sở hữu chi phí điện toán lớn cho mỗi lần kiểm chứng cấu hình. Thay vì quét tuyến tính như Grid Search hay lấy mẫu xác suất độc lập như Random Search, phương pháp này thiết lập một mô hình xấp xỉ (surrogate model) nhằm ước lượng mối tương quan phi tuyến giữa hệ thống siêu tham số và hiệu năng thực thi của mô hình [10].

3.2 Gaussian Process

Định nghĩa 3.1 (Gaussian Process). *Gaussian Process (GP) [2] là một tập hợp các biến ngẫu nhiên, trong đó bất kỳ tập con hữu hạn nào đều có phân phối Gaussian đa chiều. GP được xác định hoàn toàn bởi:*

- Hàm trung bình: $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$
- Hàm hiệp phương sai (kernel): $k(\mathbf{x}, \mathbf{x}') = \text{Cov}[f(\mathbf{x}), f(\mathbf{x}')]]$

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \quad (16)$$

Cho tập quan sát $\mathcal{D}_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, phân phối hậu nghiệm tại điểm mới $\mathbf{x}$ là Gaussian:

$$\mu_n(\mathbf{x}) = \mathbf{k}^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y} \quad (17)$$

$$\sigma_n^2(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - \mathbf{k}^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k} \quad (18)$$

trong đó $\mathbf{k} = [k(\mathbf{x}, \mathbf{x}_1), \dots, k(\mathbf{x}, \mathbf{x}_n)]^T$ là vector kernel giữa $\mathbf{x}$ và các điểm đã quan sát, $\mathbf{K}$ là ma trận kernel $n \times n$, σ_n^2 là phương sai nhiễu, và $\mathbf{y} = [y_1, \dots, y_n]^T$.

3.3 Hàm thu nhận (Acquisition Functions)

Hàm thu nhận cân bằng giữa **khai thác** (exploitation — chọn nơi $\mu(\mathbf{x})$ cao) và **khám phá** (exploration — chọn nơi $\sigma(\mathbf{x})$ cao).

3.3.1 Expected Improvement (EI)

Hàm EI tính kỳ vọng của phần cải thiện so với giá trị tốt nhất hiện tại $f^+ = \max_i y_i$:

$$\text{EI}(\mathbf{x}) = \mathbb{E}[\max(f(\mathbf{x}) - f^+, 0)] \quad (19)$$

Với phân phối hậu nghiệm Gaussian, công thức có dạng đóng:

$$\text{EI}(\mathbf{x}) = \begin{cases} (\mu(\mathbf{x}) - f^+) \Phi(z) + \sigma(\mathbf{x}) \phi(z), & \text{nếu } \sigma(\mathbf{x}) > 0 \\ 0, & \text{nếu } \sigma(\mathbf{x}) = 0 \end{cases} \quad (20)$$

trong đó:

$$z = \frac{\mu(\mathbf{x}) - f^+}{\sigma(\mathbf{x})} \quad (21)$$

$\Phi(\cdot)$ là hàm phân phối tích lũy (CDF) và $\phi(\cdot)$ là hàm mật độ xác suất (PDF) của phân phối chuẩn $\mathcal{N}(0, 1)$.

3.3.2 Probability of Improvement (PI)

$$\text{PI}(\mathbf{x}) = \Phi\left(\frac{\mu(\mathbf{x}) - f^+ - \xi}{\sigma(\mathbf{x})}\right) \quad (22)$$

với $\xi \geq 0$ là tham số điều khiển exploration–exploitation trade-off. Khi $\xi = 0$, PI hội tụ nhanh nhưng dễ kẹt tại cực trị cục bộ.

3.3.3 Lower Confidence Bound (LCB)

$$\text{LCB}(\mathbf{x}) = \mu(\mathbf{x}) - \beta \sigma(\mathbf{x}) \quad (23)$$

với $\beta > 0$ là hệ số cân bằng. Giá trị β lớn ưu tiên khám phá.

Nhận xét 3.1. Trong *HAutoML*, hàm ***gp_minimize*** từ *scikit-optimize* sử dụng mặc định EI với chiến lược ***gp_hedge*** — tự động chọn giữa EI, PI, LCB dựa trên hiệu suất tích lũy.

3.4 Xử lý mất cân bằng lớp

Hệ thống tự động phát hiện mất cân bằng lớp dựa trên ngưỡng:

$$\text{imbalanced} = \begin{cases} \text{True}, & \text{nếu } \max_c r_c - \min_c r_c > \tau \\ \text{False}, & \text{ngược lại} \end{cases} \quad (24)$$

trong đó $r_c = \frac{|C_c|}{|\mathcal{D}|}$ là tỷ lệ mẫu của lớp c , $\tau = 0.3$ là ngưỡng mặc định. Nếu mất cân bằng, sử dụng *weighted averaging*; ngược lại sử dụng *macro averaging*.

3.5 Phép nghịch đảo dấu trong hàm mục tiêu

Do hàm `gp_minimize` được thiết kế để giải bài toán tối thiểu hóa, trong khi các chỉ số hiệu năng (accuracy, F1, R^2) yêu cầu tối đa hóa, hàm mục tiêu phải được chuẩn hóa thông qua phép nghịch đảo dấu:

$$\operatorname{argmax}_{\theta \in \Theta} \text{Score}(\theta) \equiv \operatorname{argmin}_{\theta \in \Theta} [-\text{Score}(\theta)] \quad (25)$$

Ví dụ: cấu hình A đạt accuracy = 0.90 và cấu hình B đạt accuracy = 0.85 sẽ được ánh xạ thành -0.90 và -0.85 . Thuật toán tối thiểu hóa tự động định danh -0.90 là kết quả ưu việt hơn, bảo đảm tính nhất quán với mục tiêu tìm kiếm điểm số cao nhất.

3.6 Giả mã

Thuật toán Bayesian Optimization trong HAutoML sử dụng các thủ tục phụ trợ: phát hiện mất cân bằng lớp (Thuật toán 8), chuyển đổi không gian tìm kiếm (Thuật toán 9), tải/lưu trạng thái optimizer cho warm start (Thuật toán 10), và tối ưu hóa trên một grid đơn lẻ (Thuật toán 11).

3.6.1 Các thủ tục phụ trợ

Algorithm 8 Phát hiện mất cân bằng lớp (DETECTCLASSIMBALANCE)

Require: $\mathbf{y}$ (mảng nhãn), $\tau = 0.3$ (ngưỡng mất cân bằng)

Ensure: `is_imbalanced` $\in \{\text{true}, \text{false}\}$

- 1: `counts` $\leftarrow$ COUNTER($\mathbf{y}$); $n \leftarrow |\mathbf{y}|$
 - 2: `ratios` $\leftarrow \{c : \text{count}/n \text{ cho mỗi } (c, \text{count}) \in \text{counts}\}$
 - 3: **return** `max(ratios) - min(ratios) >  $\tau$`
-

Algorithm 9 Chuyển đổi không gian tìm kiếm sang định dạng skopt (CONVERTSEARCHSPACE)

Require: `param_grid` (dict: tên tham số $\rightarrow$ giá trị/dimension)

Ensure: `dims` (danh sách skopt dimensions), `names` (danh sách tên)

- 1: `dims` $\leftarrow []$; `names` $\leftarrow []$
 - 2: **for** mỗi `(name, values)` $\in$ `param_grid` **do**
 - 3: **if** `values` là Real/Integer/Categorical (skopt dimension) **then**
 - 4: Gán tên nếu chưa có; `dims.append(values)`
 - 5: **else if** `values` là list hoặc tuple **then**
 - 6: `dims.append(CATEGORICAL(values, name))`
 - 7: **else**
 - 8: **raise** ValueError ▷ Kiểu không hợp lệ
 - 9: **end if**
 - 10: `names.append(name)`
 - 11: **end for**
 - 12: **return** `dims`, `names`
-

Algorithm 10 Tải trạng thái optimizer cho warm start (LOADOPTIMIZERSTATE)

Require: h_{space} (hash không gian tìm kiếm), model_name, state_store, enabled

Ensure: $\mathbf{x}_0, \mathbf{y}_0$ (điểm + giá trị từ lần chạy trước, hoặc None)

```
1: if enabled = false then
2:   return None, None
3: end if
4: key  $\leftarrow$  model_name ||  $h_{\text{space}}$ 
5: if key  $\in$  state_store và state_store[key]. $h_{\text{space}}$  =  $h_{\text{space}}$  then
6:    $\mathbf{x}_0 \leftarrow$  state_store[key].x_iters;    $\mathbf{y}_0 \leftarrow$  state_store[key].func_vals
7:   if  $\mathbf{x}_0 \neq \emptyset$  và  $\mathbf{y}_0 \neq \emptyset$  then
8:     return  $\mathbf{x}_0, \mathbf{y}_0$                                  $\triangleright$  Warm start với dữ liệu lịch sử
9:   end if
10: end if
11: return None, None
```

Algorithm 11 Tối ưu hóa Bayesian trên một grid đơn lẻ (SEARCHSINGLEGRID)

Require: $\mathcal{M}$, param_grid (một grid đơn lẻ), $(\mathbf{X}, \mathbf{y})$, k , n_{calls} , n_{init} , $\alpha(\cdot)$, T_{max}

Require: es_enabled, es_patience p , ϵ (convergence threshold), warm start config

Ensure: θ^* , s^* , all_scores*, cv_results

- 1: dims, names $\leftarrow$ CONVERTSEARCHSPACE(param_grid)
- 2: avg_method $\leftarrow$ weighted nếu DETECTCLASSIMBALANCE($\mathbf{y}$), macro nếu không
- 3: $\mathcal{H} \leftarrow []$; best_all_scores $\leftarrow$ None; $t_{\text{last}} \leftarrow$ TIME()

// Định nghĩa hàm mục tiêu

- 4: **function** OBJECTIVE(θ)
- 5: $\mathcal{M}.\text{set_params}(\theta)$
- 6: cv_scores $\leftarrow$ CROSSVALIDATE($\mathcal{M}, \mathbf{X}, \mathbf{y}, k$, scoring)
- 7: last_metrics $\leftarrow$ {key : cv_scores[test_key] cho mỗi key}
- 8: **return** -last_metrics[metric_sort] ▷ Đảo dấu: max $\rightarrow$ min
- 9: **end function**

// Định nghĩa callback cho mỗi iteration

- 10: **function** ONSTEP(res)
- 11: $t_{\text{iter}} \leftarrow$ TIME() - t_{last} ; $t_{\text{last}} \leftarrow$ TIME()
- 12: $s_{\text{current}} \leftarrow$ -res.func_vals[-1]; $s_{\text{best}} \leftarrow$ -res.fun
- 13: Cập nhật cv_results, best_all_scores; $\mathcal{H}.\text{append}(s_{\text{best}})$
- 14: *// Kiểm tra time limit (EMA proactive)*
- 15: **if** SHOULDSTARTNEXTITERATION($t_{\text{iter}}, T_{\text{max}}, \dots$) = false **then** ▷ Dừng gp_minimize
- 16: **return** true
- 17: **end if**
- 18: *// Kiểm tra early stopping (chỉ khi không có time limit)*
- 19: **if** $T_{\text{max}} = \text{None}$ và es_enabled và $|\mathcal{H}| \geq p$ **then**
- 20: **if** |Set($\mathcal{H}[-p:]$)| = 1 **then** ▷ Dừng: không cải thiện
- 21: **return** true
- 22: **end if**
- 23: **if** $|\mathcal{H}| > 1$ và $|\mathcal{H}[-1] - \mathcal{H}[-2]| < \epsilon$ **then** ▷ Dừng: hội tụ
- 24: **return** true
- 25: **end if**
- 26: **end if** ▷ Tiếp tục tối ưu hóa
- 27: **return** false
- 28: **end function**

// Warm start: tải trạng thái từ lần chạy trước

- 29: $h \leftarrow$ MD5(dims)[-8]
- 30: $\mathbf{x}_0, \mathbf{y}_0 \leftarrow$ LOADOPTIMIZERSTATE($h, \mathcal{M}.\text{name}, \dots$)
- 31: **if** $\mathbf{x}_0 \neq \text{None}$ **then** ▷ Giảm 50% điểm khởi tạo
- 32: $n_{\text{init}} \leftarrow \max(1, \lfloor n_{\text{init}}/2 \rfloor)$
- 33: **end if**

// Thực thi tối ưu hóa Bayesian

- 34: result $\leftarrow$ GP_MINIMIZE(OBJECTIVE, dims, n_{calls} , n_{init} , α , callback = [ONSTEP], $\mathbf{x}_0, \mathbf{y}_0$)
- 35: SAVEOPTIMIZERSTATE(result, $h, \mathcal{M}.\text{name}, \dots$) ▷ Lưu cho warm start tương lai
- 36: $\theta^* \leftarrow$ {names[i] : result.x[i]}; $s^* \leftarrow$ -result.fun
- 37: **return** θ^* , s^* , best_all_scores, cv_results

3.6.2 Thuật toán chính

Algorithm 12 Bayesian Optimization trong HAutoML — hỗ trợ multi-grid (sử dụng gp_minimize)

Require: Mô hình $\mathcal{M}$, không gian tham số Θ (dict hoặc list-of-dicts), dữ liệu $\mathcal{D} = (\mathbf{X}, \mathbf{y})$, số fold k
Require: n_{calls} , n_{init} , hàm thu nhận $\alpha(\cdot)$, T_{max} (tùy chọn)
Ensure: θ^* , s^* , all_scores^* , cv_results , $\text{time_limit_reached}$

```

1: STARTTIMER(); state_store  $\leftarrow \{\}$ 
   // Chuẩn hóa param_grid về list-of-dicts
2: if  $\Theta$  là dict then
3:    $\Theta_{\text{list}} \leftarrow [\Theta]$ 
4: else
5:    $\Theta_{\text{list}} \leftarrow \Theta$ 
6: end if
   // Chạy tối ưu hóa trên từng grid
7: all_results  $\leftarrow []$ 
8: for mỗi grid  $\in \Theta_{\text{list}}$  do
9:   if grid =  $\emptyset$  then
10:    result  $\leftarrow \text{EVALUATEDEFAULTPARAMS}(\mathcal{M}, \mathbf{X}, \mathbf{y})$  ▷ Model không có hyperparameters
11:   else
12:    result  $\leftarrow \text{SEARCHSINGLEGRID}(\mathcal{M}, \text{grid}, \mathbf{X}, \mathbf{y}, k, n_{\text{calls}}, n_{\text{init}}, \alpha, T_{\text{max}}, \dots)$ 
13:   end if
14:   all_results.append(result)
15: end for
   // Chọn kết quả tốt nhất từ tất cả các grid
16:  $i^* \leftarrow \text{argmax}_i \text{all\_results}[i].s^*$ 
17:  $\theta^*, s^*, \text{all\_scores}^* \leftarrow \text{all\_results}[i^*]$ 
18:  $\text{cv\_results} \leftarrow \text{COMBINECVRESULTS}(\text{all\_results})$ 
   // Chuyển đổi kiểu numpy  $\rightarrow$  Python gốc
19: return  $\theta^*$ ,  $s^*$ ,  $\text{all\_scores}^*$ ,  $\text{cv\_results}$ ,  $\text{time\_limit\_reached}$ 

```

Nhận xét 3.2. Khi T_{max} được xác lập, hệ thống ưu tiên tối ưu hóa tài nguyên theo dòng thời gian vật lý và **không** áp dụng early stopping. Ngược lại, khi không có ràng buộc thời hạn, early stopping đóng vai trò bộ lọc thông minh nhằm ngắt tiến trình nếu hiệu năng không cải thiện đáng kể.

3.7 Phân tích độ phức tạp

- **Thời gian:** $\mathcal{O}(n_{\text{calls}} \cdot [n^3 + k \cdot T(\mathcal{D}, \theta)])$, trong đó n^3 là chi phí đảo ma trận kernel (n là số quan sát tích lũy).
- **Không gian:** $\mathcal{O}(n^2)$ cho ma trận kernel $\mathbf{K}$.

4 Genetic Algorithm — Thuật toán di truyền

4.1 Tổng quan

Thuật toán di truyền (GA) mô phỏng quá trình tiến hóa tự nhiên thông qua ba toán tử chính: **Selection** (chọn lọc), **Crossover** (lai ghép), và **Mutation** (đột biến) [4].

4.2 Mã hóa cá thể

Mỗi cá thể trong quần thể biểu diễn một tổ hợp siêu tham số $\theta = (\theta_1, \theta_2, \dots, \theta_m)$. Hệ thống hỗ trợ ba kiểu mã hóa:

- **Categorical:** $\theta_j \in \{v_1, v_2, \dots, v_n\} \rightarrow$ mã hóa dạng chỉ số $[0, n - 1]$.
- **Integer:** $\theta_j \in [\theta_j^{\min}, \theta_j^{\max}] \cap \mathbb{Z} \rightarrow$ mã hóa liên tục, làm tròn khi giải mã.
- **Continuous:** $\theta_j \in [\theta_j^{\min}, \theta_j^{\max}] \subset \mathbb{R} \rightarrow$ mã hóa trực tiếp.

4.3 Tournament Selection — Chọn lọc đấu trường

Chọn ngẫu nhiên k_t cá thể, lấy cá thể có fitness cao nhất. Xác suất cá thể có hạng i (trong P cá thể) chiến thắng:

$$P_i = \frac{(P - i + 1)^{k_t} - (P - i)^{k_t}}{P^{k_t}} \quad (26)$$

Nhận xét 4.1 (Adaptive Tournament Size). *Khi đa dạng quần thể $d < 0.1$, tournament size giảm để tăng exploration; khi $d > 0.5$, tournament size tăng để tăng exploitation:*

$$k_t^{\text{adapt}} = \begin{cases} \max(2, k_t - 1), & \text{nếu } d < 0.1 \\ \min\left(\frac{P}{2}, k_t + 1\right), & \text{nếu } d > 0.5 \\ k_t, & \text{ngược lại} \end{cases} \quad (27)$$

4.4 BLX- α Crossover — Lai ghép

Toán tử BLX- α [5] mở rộng miền tìm kiếm ra ngoài khoảng giữa hai cha mẹ. Cho hai cha mẹ p_1, p_2 (với $p_1 \leq p_2$) tại chiều j :

$$\begin{aligned} I &= p_2 - p_1 \\ \text{lower} &= p_1 - \alpha \cdot I \\ \text{upper} &= p_2 + \alpha \cdot I \\ c_j &\sim \mathcal{U}(\text{lower}, \text{upper}) \end{aligned} \quad (28)$$

với $\alpha \in [0, 0.5]$ (mặc định $\alpha = 0.5$). Kết quả được giới hạn trong $[\theta_j^{\min}, \theta_j^{\max}]$.

Nhận xét 4.2 (Adaptive Crossover Rate). *Khi đa dạng quần thể $d < 0.1$, tỷ lệ lai ghép tự động tăng 20%:*

$$p_c^{\text{adapt}} = \begin{cases} \min(1.0, p_c \times 1.2), & \text{nếu } d < 0.1 \\ p_c, & \text{ngược lại} \end{cases} \quad (29)$$

Đối với tham số categorical, sử dụng **hoán đổi ngẫu nhiên** (random swap) với xác suất 0.5.

4.5 Adaptive Gaussian Mutation — Đột biến thích nghi

Tỷ lệ đột biến giảm dần theo thế hệ để chuyển từ khám phá sang tinh chỉnh:

$$p_m(g) = p_m^{(0)} \cdot \left(1 - \frac{g}{G}\right) \quad (30)$$

với $p_m^{(0)}$ là tỷ lệ đột biến ban đầu, g là thế hệ hiện tại, G là tổng số thế hệ.

Đối với tham số liên tục/nguyên, cường độ đột biến cũng giảm thích nghi:

$$\sigma_{\text{mut}}(g) = (\theta_j^{\max} - \theta_j^{\min}) \cdot 0.2 \cdot \left(1 - \frac{g}{G+1}\right) \quad (31)$$

$$\theta'_j = \text{clip}(\theta_j + \mathcal{N}(0, \sigma_{\text{mut}}^2), \theta_j^{\min}, \theta_j^{\max}) \quad (32)$$

Đối với tham số categorical, đột biến thay thế giá trị hiện tại bằng một giá trị ngẫu nhiên khác từ tập giá trị có thể.

4.6 Elitism

Giữ lại e cá thể tốt nhất mà không qua chọn lọc/đột biến:

$$\text{Elite}_g = \text{top-}e(\text{Population}_g, \text{fitness}) \quad (33)$$

4.7 Độ đa dạng quần thể

4.7.1 Phương pháp $O(n)$ — Variance-based (quần thể lớn, $P \geq 20$)

$$d_j = \begin{cases} \frac{|\text{unique}(\theta_j^{(1)}, \dots, \theta_j^{(P)})|}{P}, & \text{categorical} \\ \min\left(1, 4 \cdot \text{Var}\left(\frac{\theta_j^{(i)} - \theta_j^{\min}}{\theta_j^{\max} - \theta_j^{\min}}\right)\right), & \text{continuous/integer} \end{cases} \quad (34)$$

$$d = \frac{1}{m} \sum_{j=1}^m d_j \quad (35)$$

4.7.2 Phương pháp $O(n^2)$ — Pairwise (quần thể nhỏ, $P < 20$)

$$d = \frac{2}{P(P-1)} \sum_{i < j} \frac{1}{m} \sum_{l=1}^m \delta(x_i^{(l)}, x_j^{(l)}) \quad (36)$$

trong đó:

$$\delta(a, b) = \begin{cases} \mathbb{K}[a \neq b], & \text{categorical} \\ \frac{|a - b|}{\theta_l^{\max} - \theta_l^{\min}}, & \text{continuous/integer} \end{cases} \quad (37)$$

4.8 Diversity Injection — Tiêm đa dạng

Khi quần thể trì trệ ($d < 0.05$ và không cải thiện trong 3 thế hệ liên tiếp), thay thế 20% cá thể yếu nhất bằng cá thể ngẫu nhiên mới.

4.9 Giải mã

Thuật toán Genetic Algorithm trong HAutoML sử dụng hệ thống các thủ tục phụ trợ phức tạp bao gồm: mã hóa tham số (Thuật toán 13), giải mã cá thể (Thuật toán 14), khởi tạo thông minh (Thuật toán 15), đánh giá fitness (Thuật toán 16), chọn lọc tournament thích nghi (Thuật toán 17), lai ghép BLX- α (Thuật toán 18), đột biến Gaussian thích nghi (Thuật toán 19), tính đa dạng quần thể (Thuật toán 20), tiêm đa dạng (Thuật toán 21), và tạo thế hệ mới (Thuật toán 22).

4.9.1 Các thủ tục phụ trợ

Algorithm 13 Mã hóa lưới tham số (ENCODEPARAMETERS)

Require: param_grid (dict: tên $\rightarrow$ danh sách giá trị)

Ensure: bounds (list), types (list), names (list), original_values (dict)

```
1: bounds  $\leftarrow []$ ; types  $\leftarrow []$ ; names  $\leftarrow []$ ; original_values  $\leftarrow \{\}$ 
2: for mỗi (name, values)  $\in$  param_grid do
3:   names.append(name); original_values[name]  $\leftarrow$  values
4:   if Tất cả  $v \in$  values là float then
5:     types.append(continuous); bounds.append(min(values), max(values))
6:   else if Tất cả  $v \in$  values là int then
7:     types.append(integer); bounds.append(min(values), max(values))
8:   else
9:     types.append(categorical); bounds.append(0, |values| - 1)
10:  end if
11: end for
12: return bounds, types, names, original_values
```

Algorithm 14 Giải mã cá thể về tham số gốc (DECODEINDIVIDUAL)

Require: $\mathbf{x}$ (vector số thực), names, types, original_values

Ensure: θ (dict: tên tham số $\rightarrow$ giá trị gốc)

```
1:  $\theta \leftarrow \{\}$ 
2: for  $j = 0$  đến  $|\text{names}| - 1$  do
3:   if  $\text{types}[j] = \text{categorical}$  then
4:      $\text{idx} \leftarrow \min(\lfloor x_j \rfloor, |\text{original\_values}[\text{names}[j]]| - 1)$   $\triangleright$  Làm tròn + clip
5:      $\theta[\text{names}[j]] \leftarrow \text{original\_values}[\text{names}[j]][\text{idx}]$ 
6:   else if  $\text{types}[j] = \text{integer}$  then
7:      $\theta[\text{names}[j]] \leftarrow \lfloor x_j \rfloor$   $\triangleright$  Làm tròn về integer
8:   else
9:      $\theta[\text{names}[j]] \leftarrow x_j$   $\triangleright$  Continuous: giữ nguyên
10:  end if
11: end for
12: return  $\theta$ 
```

Algorithm 15 Khởi tạo thông minh quần thể (SMARTINITIALIZATION)

Require: P (kích thước quần thể), bounds, types, original_values

Ensure: Pop_0 (danh sách P cá thể)

```
1:  $\text{Pop}_0 \leftarrow []$ 
   // Cá thể 1: Tất cả giá trị giữa (median)
2:  $\text{Pop}_0.\text{append}\left(\left[\frac{\text{lo}_j + \text{hi}_j}{2}\right]_{j=1}^m\right)$ 
   // Cá thể 2: Tất cả giá trị nhỏ nhất (lower bound)
3:  $\text{Pop}_0.\text{append}\left([\text{lo}_j]_{j=1}^m\right)$ 
   // Cá thể 3: Tất cả giá trị lớn nhất (upper bound)
4:  $\text{Pop}_0.\text{append}\left([\text{hi}_j]_{j=1}^m\right)$ 
   // Các cá thể còn lại: ngẫu nhiên với clip theo bounds
5: while  $|\text{Pop}_0| < P$  do
6:    $\mathbf{x} \leftarrow [\mathcal{U}(\text{lo}_j, \text{hi}_j)]_{j=1}^m$ 
7:   for  $j = 1$  đến  $m$  do
8:     if  $\text{types}[j] \in \{\text{integer}, \text{categorical}\}$  then
9:        $x_j \leftarrow \lfloor x_j \rfloor$   $\triangleright$  Làm tròn cho integer/categorical
10:    end if
11:  end for
12:   $\text{Pop}_0.\text{append}(\mathbf{x})$ 
13: end while
14: return  $\text{Pop}_0$ 
```

Algorithm 16 Đánh giá fitness cá thể có cache (EVALUATEINDIVIDUAL)

Require: Cá thể $\mathbf{x}$, mô hình $\mathcal{M}$, dữ liệu $(\mathbf{X}, \mathbf{y})$, k -fold, scoring, cache

Ensure: result (fitness + all_scores + params)

```
1:  $\theta \leftarrow \text{DECODEINDIVIDUAL}(\mathbf{x}, \dots)$ 
2:  $h \leftarrow \text{GETPARAMSHASH}(\mathcal{M}, \theta)$ 
3: if  $h \in \text{cache}$  then
4:   return cache[ $h$ ] ▷ Cache hit
5: end if
6:  $\mathcal{M}.\text{set\_params}(\theta)$ 
7:  $\text{cv\_scores} \leftarrow \text{CROSSVALIDATE}(\mathcal{M}, \mathbf{X}, \mathbf{y}, k, \text{scoring})$ 
8:  $\text{fitness} \leftarrow \text{cv\_scores}[\text{metric\_sort}]$ 
9:  $\text{result} \leftarrow (\text{fitness}, \{\text{key} : \text{cv\_scores}[\text{key}]\}, \theta)$ 
10: cache[ $h$ ]  $\leftarrow \text{result}$ 
11: return result
    // Nếu xảy ra lỗi: trả về fitness =  $-\infty$  (tự động loại bỏ)
```

Algorithm 17 Chọn lọc tournament thích nghi (ADAPTIVETOURNAMENTSELECTION)

Require: Quần thể Pop (đã sắp xếp), fitness $\mathbf{f}$, k_{base} , đa dạng d

Ensure: Cá thể được chọn $\mathbf{x}^*$

```
    // Adaptive tournament size: tăng áp lực chọn lọc khi đa dạng cao
1: if  $d > 0.3$  then
2:    $k_t \leftarrow \min(|\text{Pop}|, k_{\text{base}} + 1)$ 
3: else if  $d < 0.1$  then
4:    $k_t \leftarrow \max(2, k_{\text{base}} - 1)$  ▷ Giảm áp lực khi đa dạng thấp
5: else
6:    $k_t \leftarrow k_{\text{base}}$ 
7: end if
8: pool  $\leftarrow \text{RANDOMSAMPLE}(\text{Pop}, k_t)$ 
9: return  $\text{argmax}_{\mathbf{x} \in \text{pool}} f(\mathbf{x})$ 
```

Algorithm 18 Lai ghép BLX- α chi tiết (BLXALPHACROSSOVER)

Require: Cha mẹ $\mathbf{p}_1, \mathbf{p}_2$, tỷ lệ lai ghép p_c , đa dạng d , bounds, types, $\alpha = 0.5$

Ensure: Con cái $\mathbf{c}_1, \mathbf{c}_2$

```
// Adaptive crossover rate
1: if  $d < 0.1$  then
2:    $p_c^{\text{adapt}} \leftarrow \min(1.0, p_c \times 1.2)$  ▷ Tăng 20% khi đa dạng thấp
3: else
4:    $p_c^{\text{adapt}} \leftarrow p_c$ 
5: end if
6: if  $\mathcal{U}(0, 1) > p_c^{\text{adapt}}$  then
7:   return  $\mathbf{p}_1, \mathbf{p}_2$  ▷ Không lai ghép
8: end if
9:  $\mathbf{c}_1, \mathbf{c}_2 \leftarrow \text{copy}(\mathbf{p}_1), \text{copy}(\mathbf{p}_2)$ 
10: for  $j = 0$  đến  $m - 1$  do
11:   if types[ $j$ ] = categorical then
12:     if  $\mathcal{U}(0, 1) < 0.5$  then
13:        $c_{1,j}, c_{2,j} \leftarrow c_{2,j}, c_{1,j}$  ▷ Hoán đổi ngẫu nhiên
14:     end if
15:   else
16:     // BLX- $\alpha$  cho continuous/integer
17:      $I \leftarrow |p_{2,j} - p_{1,j}|$ 
18:      $\text{lo} \leftarrow \min(p_{1,j}, p_{2,j}) - \alpha \cdot I$ ;  $\text{hi} \leftarrow \max(p_{1,j}, p_{2,j}) + \alpha \cdot I$ 
19:      $c_{1,j} \leftarrow \text{CLIP}(\mathcal{U}(\text{lo}, \text{hi}), \text{bound}_j)$ 
20:      $c_{2,j} \leftarrow \text{CLIP}(\mathcal{U}(\text{lo}, \text{hi}), \text{bound}_j)$ 
21:     if types[ $j$ ] = integer then
22:        $c_{1,j} \leftarrow \lfloor c_{1,j} \rfloor$ ;  $c_{2,j} \leftarrow \lfloor c_{2,j} \rfloor$ 
23:     end if
24:   end if
25: end for
26: return  $\mathbf{c}_1, \mathbf{c}_2$ 
```

Algorithm 19 Đột biến Gaussian thích nghi (ADAPTIVEMUTATE)

Require: Cá thể $\mathbf{x}$, thể hệ g , tổng thể hệ G , $p_m^{(0)}$, bounds, types

Ensure: $\mathbf{x}'$ (cá thể sau đột biến)

```
1:  $\mathbf{x}' \leftarrow \text{copy}(\mathbf{x})$ 
   // Adaptive mutation rate: giảm theo thể hệ
2:  $p_m(g) \leftarrow p_m^{(0)} \cdot (1 - g/G)$  ▷ Exploration → exploitation
3: for  $j = 0$  đến  $m - 1$  do
4:   if  $\mathcal{U}(0, 1) < p_m(g)$  then
5:     if  $\text{types}[j] = \text{categorical}$  then
6:        $x'_j \leftarrow \mathcal{U}_{\text{int}}(\text{lo}_j, \text{hi}_j)$  ▷ Chọn ngẫu nhiên index mới
7:     else
8:       // Gaussian mutation với cường độ thích nghi
9:        $\sigma \leftarrow (\text{hi}_j - \text{lo}_j) \times 0.2 \times (1 - g/(G + 1))$ 
10:       $x'_j \leftarrow \text{CLIP}(x_j + \mathcal{N}(0, \sigma^2), \text{lo}_j, \text{hi}_j)$ 
11:      if  $\text{types}[j] = \text{integer}$  then
12:         $x'_j \leftarrow \lfloor x'_j \rfloor$ 
13:      end if
14:    end if
15:  end for
16: return  $\mathbf{x}'$ 
```

Algorithm 20 Tính đa dạng quần thể — auto-switch (CALCULATEDIVERSITY)

Require: Quần thể Pop, bounds, types, ngưỡng $P_{\text{thresh}} = 20$

Ensure: $d \in [0, 1]$

```
1: if  $|\text{Pop}| \geq P_{\text{thresh}}$  then
    // Phương pháp nhanh  $O(n)$  — variance-based
2:   for  $j = 0$  đến  $m - 1$  do
3:     if  $\text{types}[j] = \text{categorical}$  then
4:        $d_j \leftarrow |\text{unique}(\{x_j^{(i)}\}_{i=1}^P)|/P$ 
5:     else
6:        $d_j \leftarrow \min\left(1, 4 \cdot \text{Var}\left(\frac{x_j^{(i)} - \text{lo}_j}{\text{hi}_j - \text{lo}_j}\right)\right)$ 
7:     end if
8:   end for
9:   return  $\frac{1}{m} \sum_{j=0}^{m-1} d_j$ 
10: else
    // Phương pháp chính xác  $O(n^2)$  — pairwise Hamming
11:    $D \leftarrow 0$ ; count  $\leftarrow 0$ 
12:   for mỗi cặp  $(i, k)$  với  $i < k$  do
13:     for  $j = 0$  đến  $m - 1$  do
14:       if  $\text{types}[j] = \text{categorical}$  then
15:          $D \leftarrow D + \mathbb{1}[x_j^{(i)} \neq x_j^{(k)}]$ 
16:       else
17:          $D \leftarrow D + \frac{|x_j^{(i)} - x_j^{(k)}|}{\text{hi}_j - \text{lo}_j}$ 
18:       end if
19:     end for
20:     count  $\leftarrow \text{count} + 1$ 
21:   end for
22:   return  $\min(1, D/(\text{count} \cdot m))$ 
23: end if
```

Algorithm 21 Tiêm đa dạng vào quần thể (INJECTDIVERSITY)

Require: Quần thể Pop, fitness $\mathbf{f}$, bounds, types, $r_{\text{inject}} = 0.2$

Ensure: Pop' (quần thể sau tiêm đa dạng)

```
1:  $n_{\text{inject}} \leftarrow \max(1, \lfloor r_{\text{inject}} \cdot |\text{Pop}| \rfloor)$ 
   // Sắp xếp theo fitness giảm dần, thay thế cá thể yếu nhất
2: indices_sorted  $\leftarrow \text{ARGSORT}(-\mathbf{f})$ 
3: for  $i = 0$  đến  $n_{\text{inject}} - 1$  do
4:   idx  $\leftarrow \text{indices\_sorted}[|\text{Pop}| - 1 - i]$ 
5:   Pop[idx]  $\leftarrow \text{CREATEINDIVIDUAL}(\text{bounds}, \text{types})$ 
6:    $\mathbf{f}[\text{idx}] \leftarrow -\infty$ 
7: end for
8: return Pop
```

▷ Cá thể yếu nhất
▷ Cá thể ngẫu nhiên mới
▷ Đánh dấu cần đánh giá lại

Algorithm 22 Tạo thế hệ mới (CREATENEXTGENERATION)

Require: Quần thể Pop_g (đã sắp xếp), fitness $\mathbf{f}$, đa dạng d , thế hệ g , tổng G

Require: e (elite size), P (pop size), p_m , p_c , bounds, types

Ensure: Pop_{g+1}

```
// Elitism: giữ lại e cá thể tốt nhất
1:  $\text{Pop}_{g+1} \leftarrow [\text{Pop}_g[0], \dots, \text{Pop}_g[e-1]]$  ▷ Đã sắp xếp giảm dần
// Sinh cá thể mới bằng chọn lọc + lai ghép + đột biến
2: while  $|\text{Pop}_{g+1}| < P$  do
3:    $\mathbf{p}_1 \leftarrow \text{ADAPTIVETOURNAMENTSELECTION}(\text{Pop}_g, \mathbf{f}, k_{\text{base}}, d)$ 
4:    $\mathbf{p}_2 \leftarrow \text{ADAPTIVETOURNAMENTSELECTION}(\text{Pop}_g, \mathbf{f}, k_{\text{base}}, d)$ 
5:    $\mathbf{c}_1, \mathbf{c}_2 \leftarrow \text{BLXALPHACROSSOVER}(\mathbf{p}_1, \mathbf{p}_2, p_c, d, \text{bounds}, \text{types})$ 
6:    $\mathbf{c}_1 \leftarrow \text{ADAPTIVEMUTATE}(\mathbf{c}_1, g, G, p_m, \text{bounds}, \text{types})$ 
7:    $\mathbf{c}_2 \leftarrow \text{ADAPTIVEMUTATE}(\mathbf{c}_2, g, G, p_m, \text{bounds}, \text{types})$ 
8:    $\text{Pop}_{g+1}.\text{extend}([\mathbf{c}_1, \mathbf{c}_2])$ 
9: end while
10:  $\text{Pop}_{g+1} \leftarrow \text{Pop}_{g+1}[:P]$  ▷ Cắt bớt nếu vượt kích thước  $P$ 
11: return  $\text{Pop}_{g+1}$ 
```

4.9.2 Thuật toán chính

Algorithm 23 Genetic Algorithm cho tối ưu hóa siêu tham số

Require: Mô hình $\mathcal{M}$, lưới tham số `param_grid`, dữ liệu $\mathcal{D} = (\mathbf{X}, \mathbf{y})$, số fold k
Require: P (kích thước quần thể), G (số thế hệ), p_m (tỷ lệ đột biến), p_c (tỷ lệ lai ghép), e (elite size)
Require: `scoring` (dict các metric), `metric_sort`, $T_{\max}$ (tùy chọn)
Require: Early stopping: `es_enabled`, `patience_max`
Ensure: θ^* , s^* , `all_scores*`, `cv_results`, `time_limit_reached`

```

// Giai đoạn 1: Mã hóa và khởi tạo
1: STARTTIMER(); cache  $\leftarrow \{\}$ ;  $t_{\text{ema}} \leftarrow \text{None}$ 
2: bounds, types, names, orig_vals  $\leftarrow \text{ENCODEPARAMETERS}(\text{param\_grid})$ 
3:  $\text{Pop}_0 \leftarrow \text{SMARTINITIALIZATION}(P, \text{bounds}, \text{types}, \text{orig\_vals})$ 
4:  $s^* \leftarrow -\infty$ ;  $\theta^* \leftarrow \emptyset$ ; patience  $\leftarrow 0$ ; cv_results  $\leftarrow []$ 
// Giai đoạn 2: Vòng lặp tiến hóa
5: for  $g = 0$  đến  $G - 1$  do
6:    $t_0 \leftarrow \text{TIME}()$ 
// Kiểm tra time limit (EMA proactive)
7:   if SHOULDSTARTNEXTITERATION( $\text{None}, T_{\max}, \dots$ ) = false then
8:     time_stopped  $\leftarrow \text{true}$ ; break
9:   end if
10:   $d \leftarrow \text{CALCULATEDIVERSITY}(\text{Pop}_g, \text{bounds}, \text{types})$ 
// Đánh giá fitness cho toàn quần thể (song song)
11:  results  $\leftarrow \text{PARALLEL}[\text{EVALUATEINDIVIDUAL}(\mathbf{x}, \mathcal{M}, \dots)$  cho mỗi  $\mathbf{x} \in \text{Pop}_g]$ 
12:  Sắp xếp  $\text{Pop}_g$  theo fitness giảm dần
13:   $s_{\text{prev}} \leftarrow s^*$ 
14:  for mỗi result  $\in$  results do
15:    if result.fitness  $> s^*$  then
16:       $s^* \leftarrow \text{result.fitness}$ ;  $\theta^* \leftarrow \text{result.params}$ ; all_scores*  $\leftarrow \text{result.all\_scores}$ 
17:    end if
18:    Cập nhật cv_results với result.params, result.all_scores
19:  end for
// Kiểm tra early stopping
20:  if  $s^* = s_{\text{prev}}$  then
21:    patience  $\leftarrow \text{patience} + 1$ 
22:  else
23:    patience  $\leftarrow 0$ 
24:  end if
25:  if  $T_{\max} = \text{None}$  và es_enabled và patience  $\geq \text{patience\_max}$  then
26:    break ▷ Early stopping
27:  end if
// Tiêm đa dạng nếu quần thể trì trệ
28:  if  $d < 0.05$  và patience  $\geq 3$  then
29:     $\text{Pop}_g \leftarrow \text{INJECTDIVERSITY}(\text{Pop}_g, \mathbf{f}, \text{bounds}, \text{types})$ 
30:  end if
// Tạo thế hệ mới
31:   $\text{Pop}_{g+1} \leftarrow \text{CREATENEXTGENERATION}(\text{Pop}_g, \mathbf{f}, d, g, G, e, P, p_m, p_c, \dots)$ 
32:  SHOULDSTARTNEXTITERATION( $\text{TIME}() - t_0, T_{\max}, \dots$ ) ▷ Cập nhật EMA
33: end for
// Giai đoạn 3: Xây dựng cv_results và ranking
34: for mỗi metric  $\in$  Keys(scoring) do
35:   cv_results[rank_test_ || metric]  $\leftarrow \text{ARGSORT}(-\text{scores}) + 1$ 
36: end for
37: Xóa cache; Chuyển đổi numpy types
38: return  $\theta^*$ ,  $s^*$ , all_scores*, cv_results, time_stopped

```

4.10 Phân tích độ phức tạp

- **Thời gian:** $\mathcal{O}(G \cdot P \cdot k \cdot T(\mathcal{D}, \theta))$, trong đó G là số thế hệ, P là kích thước quần thể.
- **Không gian:** $\mathcal{O}(P \cdot m)$ cho quần thể, cộng thêm $\mathcal{O}(C)$ cho cache (C là kích thước cache tối đa).

5 So sánh các thuật toán

5.1 Bảng so sánh chi tiết

Bảng 1: Đối chiếu các chiến lược tối ưu hóa siêu tham số

Tiêu chí	Grid Search	Bayesian Opt.	Genetic Alg.
Cơ chế xác lập	Quét tuyến tính toàn bộ	Surrogate model + hàm thu nhận	Tiến hóa: Selection, Crossover, Mutation
Độ phức tạp thời gian	$\mathcal{O}(N \cdot k \cdot T)$	$\mathcal{O}(n_c \cdot (n^3 + kT))$	$\mathcal{O}(G \cdot P \cdot k \cdot T)$
Độ phức tạp không gian	$\mathcal{O}(1)$	$\mathcal{O}(n^2)$	$\mathcal{O}(P \cdot m)$
Tham số liên tục	Kém	Xuất sắc	Tốt
Tham số rời rạc	Tốt	Tốt	Tốt
Kế thừa tri thức	Không	Có (GP posterior)	Có (Elitism)
Song song hóa	Hoàn toàn	Hạn chế	Theo thể hệ
Khám phá toàn cục	Đầy đủ	Phụ thuộc prior	Tốt
Hội tụ	N/A	Nhanh	Trung bình
Thoát local optima	N/A	Khó	Tốt
Triển khai HAutoML	Batch, cache, song song	<code>gp_minimize</code> , dừng sớm	Smart init, adaptive

Ký hiệu: $N = \prod n_j$ (tổng tổ hợp Grid Search), $n_c = \text{n_calls}$, n = số quan sát tích lũy, G = số thế hệ, P = kích thước quần thể, k = số fold CV, m = số siêu tham số, T = thời gian train/evaluate.

5.2 Hướng dẫn chọn thuật toán

Bảng 2: Khuyến nghị chọn thuật toán theo tình huống

Tình huống	Khuyến nghị
Không gian nhỏ ($N < 100$), cần đảm bảo tìm tối ưu	Grid Search
Tham số liên tục, budget đánh giá hạn chế	Bayesian Opt.
Không gian phức tạp, nhiều local optima	Genetic Alg.
Cần kết quả nhanh	Bayesian + Early stopping
Giới hạn thời gian cứng	Bất kỳ + <code>max_time</code>

5.2.1 Chi tiết các kịch bản ứng dụng

Việc lựa chọn chiến lược tối ưu hóa đóng vai trò quyết định đến sự cân bằng giữa hiệu suất và chi phí tài nguyên. Dưới đây là phân tích chi tiết cho từng kịch bản:

- **Không gian nhỏ** ($N < 100$): Khi lưới tham số đủ nhỏ (thường gặp khi tinh chỉnh 1-2 tham số rời rạc hoặc số cây trong RandomForest), **Grid Search** là lựa chọn an toàn

nhất. Khả năng duyệt cạn kết hợp với cơ chế *batch processing* song song hoàn toàn sẽ đảm bảo tìm được nghiệm tối ưu toàn cục trong thời gian chấp nhận được, mà không chịu rủi ro bỏ sót như các thuật toán mang tính xác suất.

- **Tham số liên tục & Budget đánh giá hạn chế:** Các mô hình Gradient Boosting (XGBoost, LightGBM, CatBoost) hoặc Deep Learning có nhiều siêu tham số liên tục (như `learning_rate`, `l2_regularization`) và tốn rất nhiều thời gian để huấn luyện (đánh giá). **Bayesian Optimization** là lựa chọn duy nhất hợp lý ở đây do khả năng xây dựng mô hình xấp xỉ liên tục, giúp định vị vùng cực trị với số lần đánh giá (n_{calls}) cực nhỏ (thường 30 – 50 lần).
- **Không gian phức tạp & Nhiều Local Optima:** Khi tối ưu hóa một pipeline phức tạp (bao gồm cả các tham số tiền xử lý và tham số của mô hình), không gian sẽ bị phân mảnh và chứa nhiều điểm cực tiểu cục bộ. Bayesian dễ bị kẹt (over-exploit) trong khi **Genetic Algorithm (GA)** duy trì một quần thể đa dạng, trải đều không gian và thoát local optima dễ dàng nhờ cơ chế đột biến (mutation) và tìm đa dạng tự động.
- **Cần kết quả nhanh & Giới hạn thời gian cứng:** Khi hệ thống HAutoML được triển khai trên môi trường production bị giới hạn chặt chẽ về SLA (thời gian phản hồi), tính năng `max_time` kết hợp với **EMA Proactive Stop** sẽ chặn thuật toán dừng đúng lúc. **Bayesian Optimization** kết hợp với **Early Stopping** thường cho ra một cấu hình đủ tốt (sub-optimal nhưng usable) nhanh nhất nhờ khả năng hội tụ sớm của Gaussian Process.

6 Cơ chế quản lý thời gian chung

Tất cả các thuật toán chia sẻ cơ chế quản lý thời gian từ lớp cơ sở **SearchStrategy**:

6.1 Proactive Time Limit

Sử dụng Exponential Moving Average (EMA) để ước tính thời gian iteration tiếp theo và dừng sớm nếu không đủ thời gian:

$$\hat{t}_{n+1} = 0.7 \cdot t_n + 0.3 \cdot \hat{t}_n \quad (38)$$

$$\text{continue} = \begin{cases} \text{False}, & \text{nếu } 1.2 \cdot \hat{t}_{n+1} > t_{\text{remaining}} \\ \text{True}, & \text{ngược lại} \end{cases} \quad (39)$$

Hệ số 1.2 là safety factor, dự phòng trường hợp iteration tiếp theo chậm hơn dự kiến.

6.2 Early Stopping vs. Time Limit

Hai cơ chế này hoạt động loại trừ lẫn nhau:

$$\text{apply_early_stopping} = \begin{cases} \text{False}, & \text{nếu } T_{\text{max}} \neq \text{None} \\ \text{True}, & \text{ngược lại} \end{cases} \quad (40)$$

Khi có `max_time`, hệ thống ưu tiên sử dụng hết thời gian thay vì dừng sớm, nhằm tận dụng tối đa budget thời gian.

A Bảng giải nghĩa thuật ngữ và siêu tham số

Dưới đây là bảng giải nghĩa chi tiết các thuật ngữ khoa học và tham số cấu hình hệ thống được sử dụng xuyên suốt tài liệu này cũng như trong mã nguồn triển khai của hệ thống HAutoML.

A.1 Bảng giải nghĩa các thuật ngữ chung

Bảng 3: Giải nghĩa các thuật ngữ khoa học

Tiếng Anh / Thuật ngữ	Tiếng Việt / Giải nghĩa
Hyperparameters	Siêu tham số của mô hình học máy.
Parallel Computing	Tính toán song song trên nhiều đơn vị xử lý.
Global Shared Memory	Bộ nhớ dùng chung toàn cục giữa các luồng/tiến trình.
Single-objective	Bài toán tối ưu hóa chỉ tập trung vào một mục tiêu đơn lẻ.
Multi-fidelity HPO	Tối ưu hóa siêu tham số đa độ trung thực, tăng tốc tìm kiếm bằng cách đánh giá trên tập con dữ liệu hoặc số vòng lặp ít hơn.
Multi-objective HPO	Tối ưu hóa siêu tham số đa mục tiêu, cân bằng giữa nhiều tiêu chí đối nghịch nhau như độ chính xác và thời gian suy diễn.
Distributed	Hệ thống phân tán, chạy trên nhiều nút mạng.
Distributed Computing	Tính toán phân tán trong môi trường mạng kết nối.
Parallel	Xử lý song song đồng thời nhiều tác vụ.
Network	Kết nối mạng truyền thông giữa các máy tính.
System Complexity	Độ phức tạp hệ thống phát sinh từ hạ tầng và cấu trúc phần mềm.
Hyperparameter Optimization	Tối ưu hóa siêu tham số, là quá trình tìm kiếm siêu tham số tốt nhất cho mô hình học máy.
Search Strategy	Chiến lược tìm kiếm siêu tham số được hệ thống sử dụng (ví dụ: Grid Search, Bayesian Optimization, Genetic Algorithm).
Grid Search	Phương pháp tìm kiếm dạng lưới, duyệt toàn bộ các tổ hợp tham số được định nghĩa trước.
Random Search	Phương pháp tìm kiếm ngẫu nhiên, lấy mẫu một số cấu hình trong không gian tham số thay vì duyệt toàn bộ.
Bayesian Optimization	Kỹ thuật tối ưu hóa Bayesian, sử dụng mô hình thay thế (surrogate model) để định hướng lựa chọn cấu hình tiếp theo.
Genetic Algorithm	Thuật toán di truyền, mô phỏng quá trình tiến hóa tự nhiên thông qua chọn lọc, lai ghép và đột biến.
Gaussian Process	Quá trình Gauss, thường được dùng làm mô hình thay thế trong Bayesian Optimization.
Surrogate Model	Mô hình thay thế hoặc mô hình xấp xỉ dùng để dự đoán hiệu năng của các cấu hình chưa được đánh giá thật.

Bảng 3 (tiếp theo): Giải nghĩa các thuật ngữ khoa học

Tiếng Anh / Thuật ngữ	Tiếng Việt / Giải nghĩa
Acquisition Function	Hàm lợi ích (hoặc hàm thu nhận) dùng trong Bayesian Optimization để chọn cấu hình tiếp theo cần thử nghiệm.
Expected Improvement (EI)	Kỳ vọng cải thiện, một loại hàm lợi ích dùng để ưu tiên các cấu hình có khả năng cải thiện kết quả hiện tại cao nhất.
Cross-Validation	Kiểm chứng chéo, kỹ thuật chia dữ liệu thành nhiều fold để đánh giá mô hình ổn định và khách quan hơn.
CASH	Bài toán lựa chọn đồng thời thuật toán học máy và tối ưu siêu tham số. Trong AutoML, CASH không chỉ tìm bộ tham số tốt nhất cho một mô hình cố định, mà còn so sánh nhiều thuật toán ứng viên để chọn ra mô hình và cấu hình tối ưu nhất.
Fitness Score	Điểm thích nghi đánh giá chất lượng của một cá thể trong thuật toán di truyền.
Selection	Phép toán chọn lọc các cá thể tốt để nhân bản và tạo thế hệ tiếp theo.
Crossover	Phép toán lai ghép giữa các cá thể bố mẹ để tạo ra các cá thể con mới thừa hưởng đặc tính tốt.
Mutation	Phép toán đột biến, thay đổi ngẫu nhiên một phần gen của cá thể nhằm tăng khả năng khám phá không gian tìm kiếm mới.
Elitism	Cơ chế giữ lại một số cá thể xuất sắc nhất đi thẳng sang thế hệ tiếp theo không qua biến đổi.
Population	Quần thể bao gồm tập hợp các cá thể (bộ siêu tham số) trong thuật toán di truyền.
Generation	Thế hệ trong quá trình tiến hóa lặp của thuật toán di truyền.
Warm Start	Khởi động quá trình tìm kiếm dựa trên kết quả hoặc trạng thái đã có từ các lần chạy trước đó để tiết kiệm thời gian.
Early Stopping	Dừng sớm khi quá trình huấn luyện hoặc tìm kiếm không còn ghi nhận cải thiện hiệu năng đáng kể.
Convergence Threshold	Ngưỡng hội tụ, dùng để xác định khi mức cải thiện giữa các bước lặp trở nên quá nhỏ để tiếp tục.
Batch Size	Kích thước lô xử lý, là số cấu hình được gom lại để đánh giá đồng thời trong một lần xử lý.
Parallel Evaluation	Đánh giá song song đồng thời nhiều cấu hình siêu tham số hoặc nhiều cá thể trong quần thể.
Cache Evaluation	Cơ chế lưu đệm kết quả đánh giá để tránh việc tính toán lại đối với cùng một cấu hình siêu tham số.
Time Budget	Ngân sách thời gian giới hạn được phân bổ cho toàn bộ quá trình tìm kiếm hoặc huấn luyện.
Maximum Time	Thời gian chạy tối đa cho phép của thuật toán, tương ứng với tham số <code>max_time</code> trong cấu hình.

Bảng 3 (tiếp theo): Giải nghĩa các thuật ngữ khoa học

Tiếng Anh / Thuật ngữ	Tiếng Việt / Giải nghĩa
Metric Sort	Chỉ số chính dùng để xếp hạng và lựa chọn cấu hình/mô hình tối ưu nhất (ví dụ: accuracy, f1, loss).

A.2 Bảng giải nghĩa các tham số cấu hình hệ thống và mô hình

Bảng 4: Giải nghĩa các tham số cấu hình

Tham số cấu hình	Ý nghĩa và chức năng trong hệ thống
<code>search_algorithm</code>	Tham số chọn chiến lược tìm kiếm, ví dụ <code>grid_search</code> , <code>bayesian_search</code> hoặc <code>genetic_algorithm</code> .
<code>param_grid</code>	Không gian siêu tham số cần tìm kiếm, được khai báo cấu trúc trong file YAML.
<code>metric_sort</code>	Chỉ số hiệu năng chính dùng để so sánh, xếp hạng và chọn cấu hình tối ưu.
<code>max_time</code>	Giới hạn ngân sách thời gian tối đa (tính bằng giây) cho quá trình tìm kiếm.
<code>n_jobs</code>	Số luồng hoặc tiến trình song song; giá trị <code>-1</code> nghĩa là tận dụng tối đa số nhân CPU khả dụng.
<code>cv_n_splits</code>	Số lượng phân đoạn dữ liệu (folds) trong kiểm chứng chéo k -fold.
<code>cv_shuffle</code>	Xác định xem có xáo trộn dữ liệu trước khi thực hiện chia fold hay không.
<code>cv_random_state</code>	Hạt giống ngẫu nhiên (seed) giúp quá trình chia fold dữ liệu có thể tái lập chính xác.
<code>random_state</code>	Hạt giống ngẫu nhiên chung nhằm đảm bảo khả năng tái lập kết quả của các thuật toán.
<code>error_score</code>	Giá trị trả về dự phòng khi một cấu hình tham số bị lỗi trong quá trình huấn luyện/đánh giá.
<code>parallel_evaluation</code>	Bật hoặc tắt đánh giá song song các cấu hình trong Grid Search hoặc các cá thể trong Genetic Algorithm.
<code>batch_size</code>	Số cấu hình được phân bổ vào mỗi batch thực thi trong Grid Search.
<code>auto_select_backend</code>	Cơ chế tự động lựa chọn backend song song tối ưu giữa <code>threading</code> hoặc <code>loky</code> .
<code>cache_evaluations</code>	Bật hoặc tắt việc lưu đệm kết quả đánh giá cấu hình để tránh tính toán lại.
<code>early_stopping_enabled</code>	Bật hoặc tắt cơ chế dừng sớm toàn bộ tiến trình tìm kiếm.
<code>early_stopping_score</code>	Ngưỡng điểm mục tiêu mong muốn để Grid Search có thể dừng sớm ngay khi đạt được.
<code>early_stopping_n_best</code>	Số lượng cấu hình tốt cần đạt được trước khi kích hoạt cơ chế dừng sớm.
<code>early_stopping_no_improve</code>	Số batch liên tiếp không cải thiện điểm số để kích hoạt dừng sớm trong Grid Search.
<code>n_calls</code>	Số lần gọi đánh giá hàm mục tiêu tối đa trong Bayesian Optimization.
<code>n_initial_points</code>	Số cấu hình khởi tạo ngẫu nhiên ban đầu trước khi Bayesian Optimization chuyển sang dùng mô hình dẫn hướng.

Bảng 4 (tiếp theo): Giải nghĩa các tham số cấu hình

Tham số cấu hình	Ý nghĩa và chức năng trong hệ thống
<code>acq_func</code>	Hàm thu nhận (acquisition function) sử dụng trong Bayesian Optimization; dự án hiện dùng EI (Expected Improvement).
<code>acq_optimizer</code>	Thuật toán tối ưu hóa dùng để tìm điểm cực đại của hàm thu nhận.
<code>warm_start_enabled</code>	Cho phép kế thừa và tiếp tục tối ưu hóa từ lịch sử chạy trước đó.
<code>save_optimizer_state</code>	Lưu trạng thái của bộ tối ưu hóa xuống đĩa để có thể tái sử dụng hoặc phục hồi sau này.
<code>convergence_threshold</code>	Ngưỡng kiểm tra hội tụ; nếu mức cải thiện nhỏ hơn ngưỡng này thì dừng tìm kiếm.
<code>early_stopping_patience</code>	Số vòng lặp (hoặc thế hệ) liên tiếp không cải thiện điểm số tối ưu để kích hoạt dừng sớm.
<code>population_size</code>	Kích thước quần thể (số cá thể trong một thế hệ) của Genetic Algorithm.
<code>min_population</code>	Kích thước quần thể tối thiểu khi sử dụng cơ chế điều chỉnh kích thước quần thể thích ứng.
<code>max_population</code>	Kích thước quần thể tối đa cho phép trong Genetic Algorithm.
<code>generation</code>	Số thế hệ tiến hóa tối đa cho phép của thuật toán di truyền.
<code>mutation_rate</code>	Tỷ lệ đột biến gen cơ sở của các cá thể trong Genetic Algorithm.
<code>crossover_rate</code>	Tỷ lệ lai ghép cơ sở giữa các cặp cá thể bố mẹ.
<code>elite_size</code>	Số lượng cá thể ưu tú nhất được sao chép nguyên vẹn sang thế hệ tiếp theo (Elitism).
<code>tournament_size</code>	Số lượng cá thể được chọn ngẫu nhiên tham gia mỗi lượt đấu trường chọn lọc.
<code>adaptive_tournament_size</code>	Cơ chế tự điều chỉnh kích thước đấu trường dựa trên độ đa dạng của quần thể tại thế hệ hiện tại.
<code>adaptive_population</code>	Cơ chế điều khiển tự thích ứng kích thước quần thể theo độ phức tạp không gian tìm kiếm.
<code>use_global_cache</code>	Cho phép cache kết quả đánh giá cá thể qua nhiều thế hệ của Genetic Algorithm.
<code>max_cache_size</code>	Kích thước giới hạn của bộ nhớ đệm cache kết quả đánh giá.
<code>fast_mode</code>	Chế độ tối ưu hóa tốc độ thực thi cho Genetic Algorithm thông qua giảm tần suất đánh giá phụ.
<code>ultra_fast_mode</code>	Chế độ tăng tốc cực hạn, cắt giảm tối đa các phép toán kiểm tra và đánh giá không cốt lõi.
<code>skip_diversity_check</code>	Bỏ qua bước kiểm tra độ đa dạng quần thể để đẩy nhanh tốc độ chạy.
<code>simple_crossover</code>	Sử dụng cơ chế lai ghép một điểm hoặc hai điểm đơn giản thay vì lai ghép BLX- α phức tạp để giảm chi phí.

Bảng 4 (tiếp theo): Giải nghĩa các tham số cấu hình

Tham số cấu hình	Ý nghĩa và chức năng trong hệ thống
<code>fast_diversity</code>	Phương pháp tính toán độ đa dạng nhanh dựa trên độ lệch (variance) thay vì so sánh từng cặp cá thể ($O(n)$ thay vì $O(n^2)$).
<code>alpha</code>	Hệ số pha trộn α trong toán tử lai ghép BLX- α liên tục.
<code>max_depth</code>	Độ sâu tối đa của Decision Tree hoặc các mô hình ensemble dựa trên cây (Random Forest, Gradient Boosting, XGBoost).
<code>min_samples_split</code>	Số lượng mẫu tối thiểu ở một nút để nút đó được phép phân nhánh trong mô hình cây.
<code>min_samples_leaf</code>	Số lượng mẫu tối thiểu bắt buộc phải có tại một nút lá của cây quyết định.
<code>n_estimators</code>	Số lượng cây quyết định thành phần được xây dựng trong Random Forest, Gradient Boosting hoặc XGBoost.
<code>max_features</code>	Số lượng (hoặc tỷ lệ) đặc trưng ngẫu nhiên được xét đến khi phân tách một nút trong Random Forest.
<code>n_neighbors</code>	Số lượng láng giềng gần nhất cần tham chiếu trong thuật toán KNN.
<code>weights</code>	Hàm trọng số áp dụng cho các láng giềng trong KNN (ví dụ: <code>uniform</code> hoặc <code>distance</code>).
<code>kernel</code>	Loại hàm nhân sử dụng trong thuật toán SVC (ví dụ: <code>rbf</code> , <code>linear</code> , <code>poly</code>).
<code>C</code>	Tham số điều chuẩn (regularization parameter) trong SVC hoặc mô hình Logistic Regression.
<code>gamma</code>	Hệ số kernel của các hàm nhân <code>rbf</code> , <code>poly</code> và <code>sigmoid</code> trong SVC.
<code>solver</code>	Thuật toán tối ưu hóa số học dùng để giải bài toán cực trị trong Logistic Regression.
<code>penalty</code>	Loại chuẩn hóa áp dụng chống quá khớp trong Logistic Regression (ví dụ: L1 hoặc L2).
<code>max_iter</code>	Số vòng lặp tối đa cho quá trình giải thuật tối ưu trong Logistic Regression hoặc SVC.
<code>var_smoothing</code>	Hệ số làm trơn phương sai (tránh chia cho 0) trong mô hình Gaussian Naive Bayes.
<code>learning_rate</code>	Tốc độ học (hệ số co hẹp) của mỗi cây đóng góp vào mô hình Gradient Boosting hoặc XGBoost.
<code>colsample_bytree</code>	Tỷ lệ các cột (đặc trưng) được lấy mẫu ngẫu nhiên khi xây dựng mỗi cây trong XGBoost.

Tài liệu

- [1] Snoek, J., Larochelle, H., & Adams, R. P. (2012). *Practical Bayesian Optimization of Machine Learning Algorithms*. In Advances in Neural Information Processing Systems (NeurIPS), 25, 2951–2959.
- [2] Rasmussen, C. E. & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- [3] Bergstra, J. & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13(10), 281–305.
- [4] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [5] Eshelman, L. J. & Schaffer, J. D. (1993). *Real-coded genetic algorithms and interval-schemata*. In L. D. Whitley (Ed.), Foundations of Genetic Algorithms 2, 187–202. Morgan Kaufmann.
- [6] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- [7] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [8] Mockus, J., Tiesis, V., & Zilinskas, A. (1978). *The Application of Bayesian Methods for Seeking the Extremum*. In Towards Global Optimization, 2, 117–129. Elsevier.
- [9] Jones, D. R., Schonlau, M., & Welch, W. J. (1998). *Efficient Global Optimization of Expensive Black-Box Functions*. Journal of Global Optimization, 13(4), 455–492.
- [10] Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. (2016). *Taking the Human Out of the Loop: A Review of Bayesian Optimization*. Proceedings of the IEEE, 104(1), 148–175.